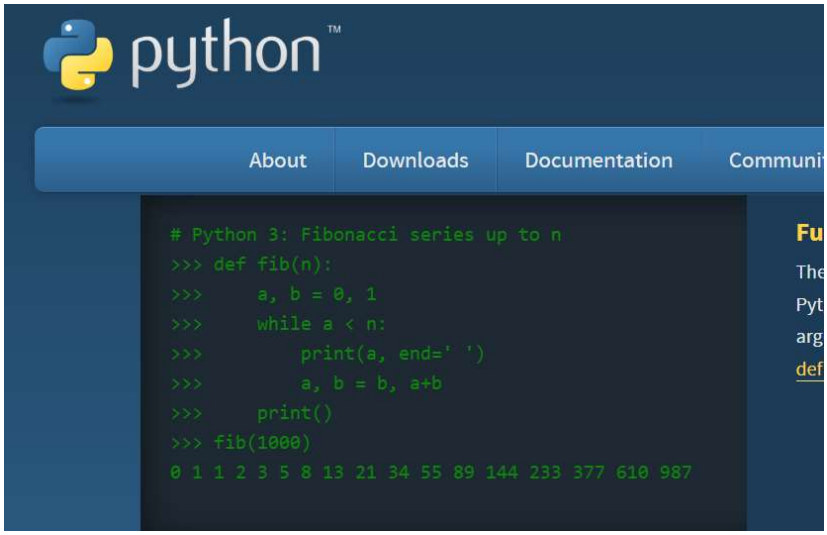
MYSQL Database Connection using Python

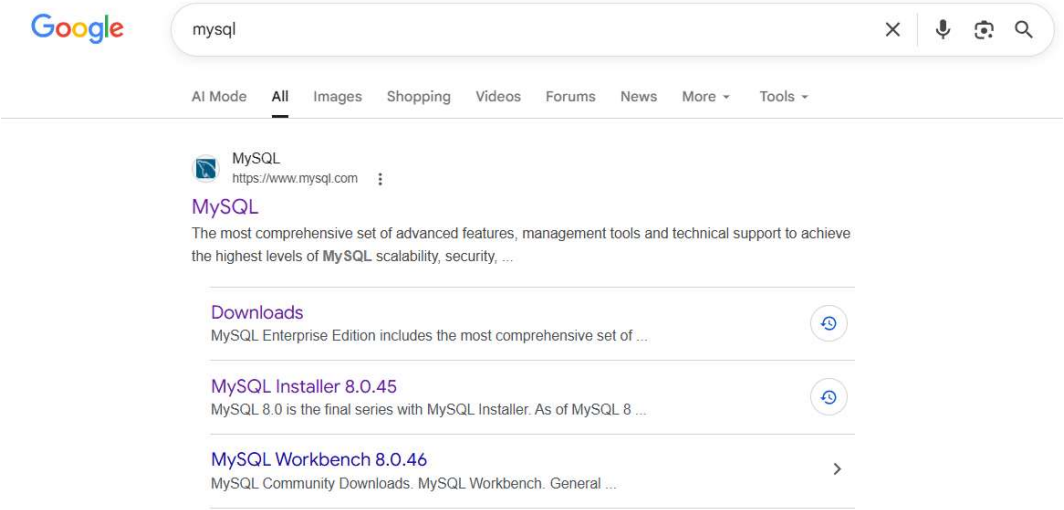
(1) Install Python



(2) Open cmd and check python version

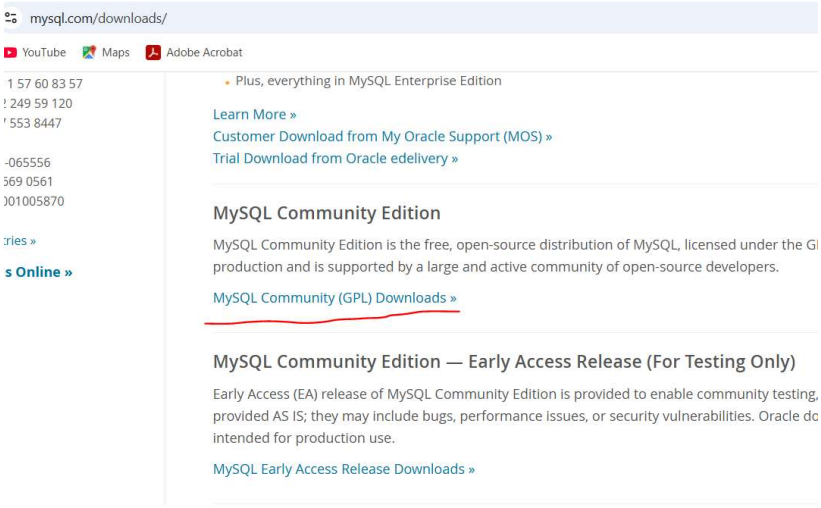
(3) Install MySQL

a. Search “mysql” in google

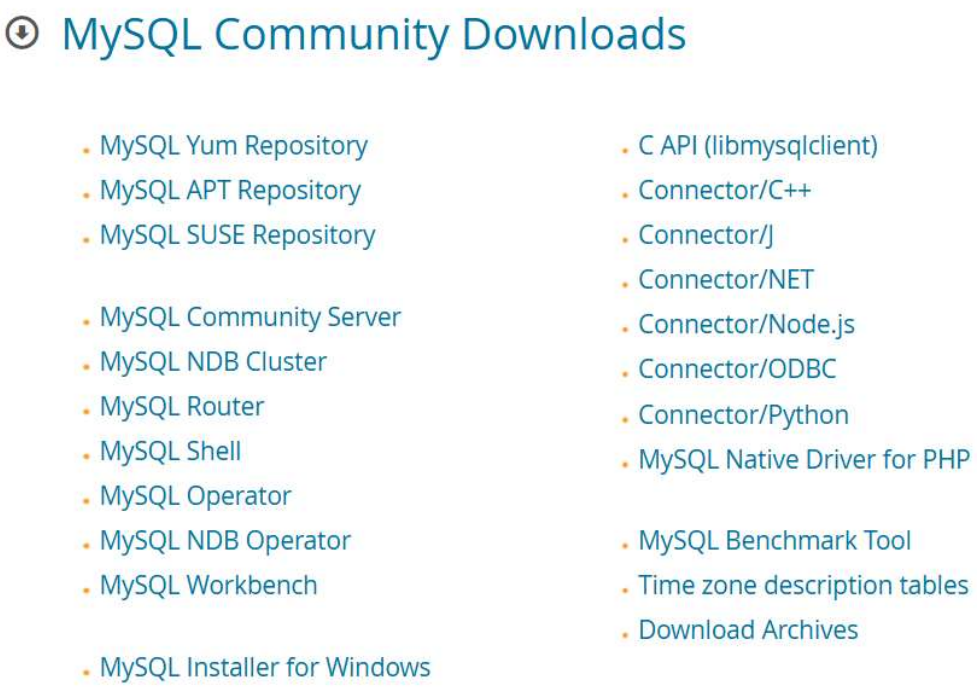


b. Click on Mysql as shown earlier pic. Below screen will appear.

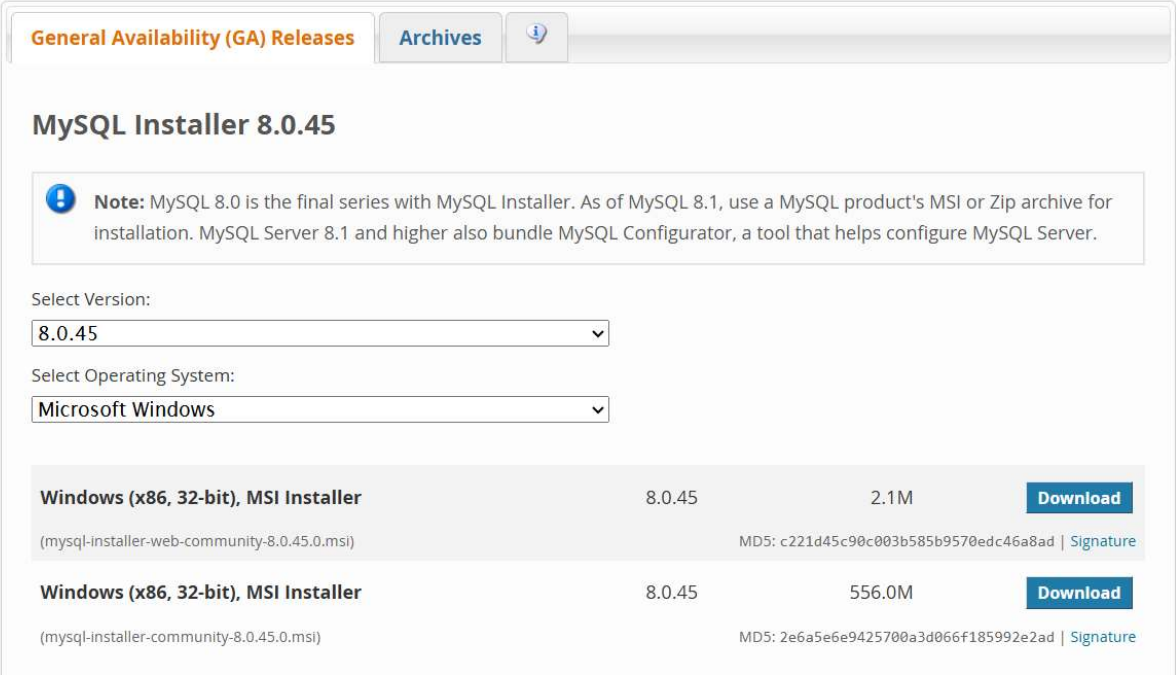
c. Click on Downloads



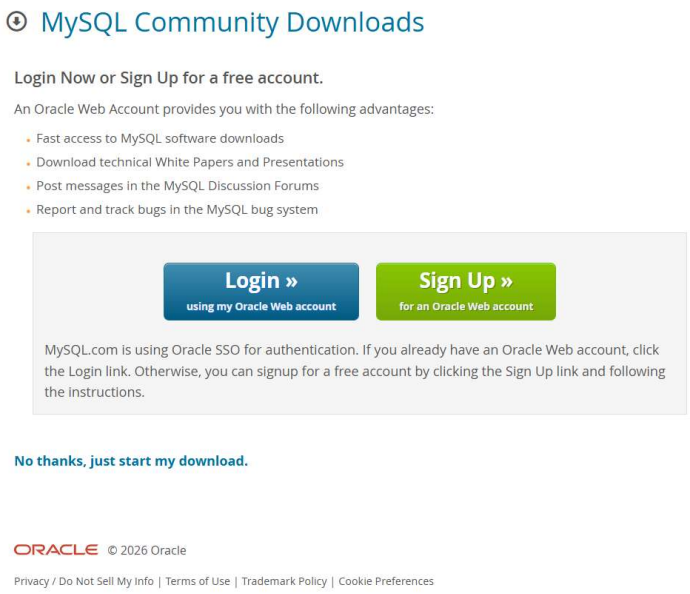
d. Click on “Mysql Community(GPL) downloads.



e. Now click on Mysql installer for windows



f. Click on 2nd installer, 556 mb one.

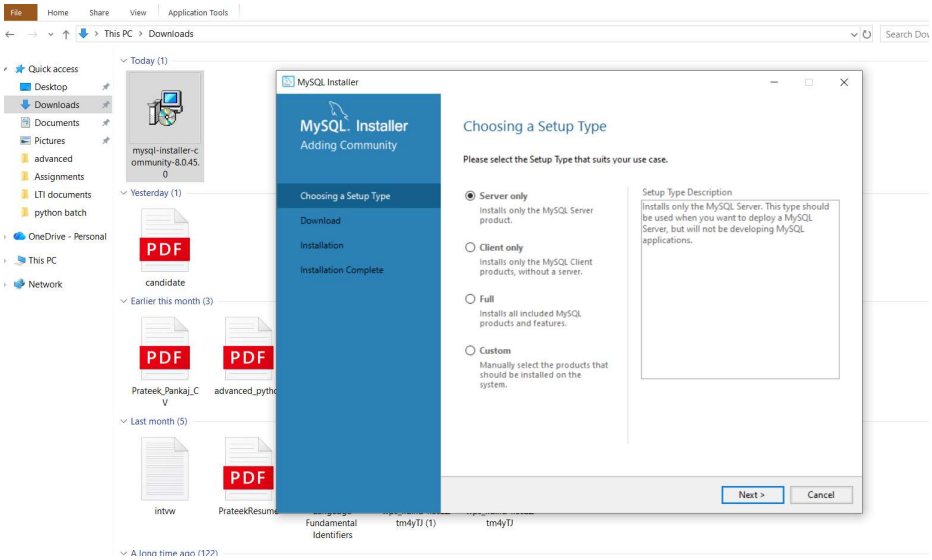


g. Click on “No Thanks….just start my download.

h. After clicking, Downloading……Started.

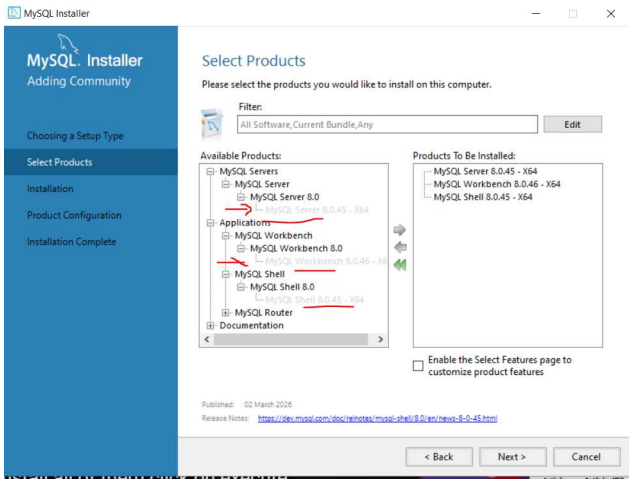
i. Double click on downloaded software.

j. After few second/minute, UI will appear.

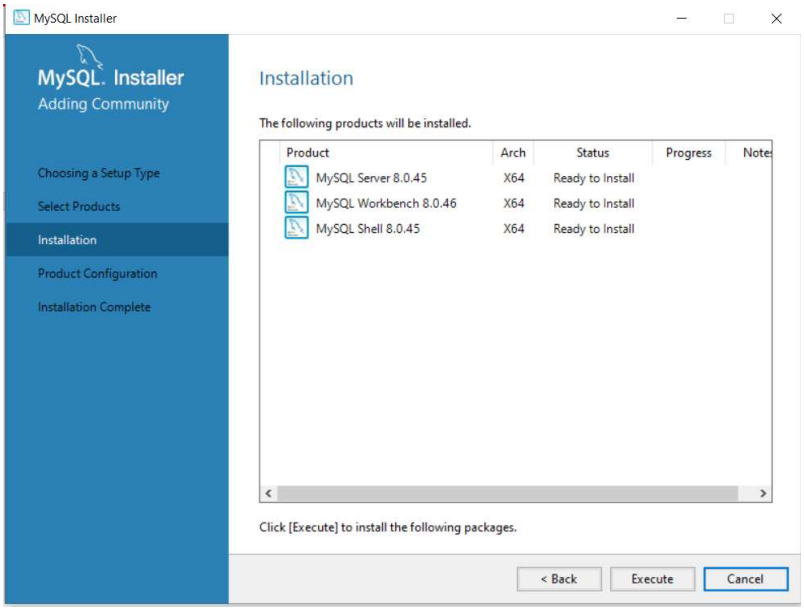


k. Click on custom and click next.

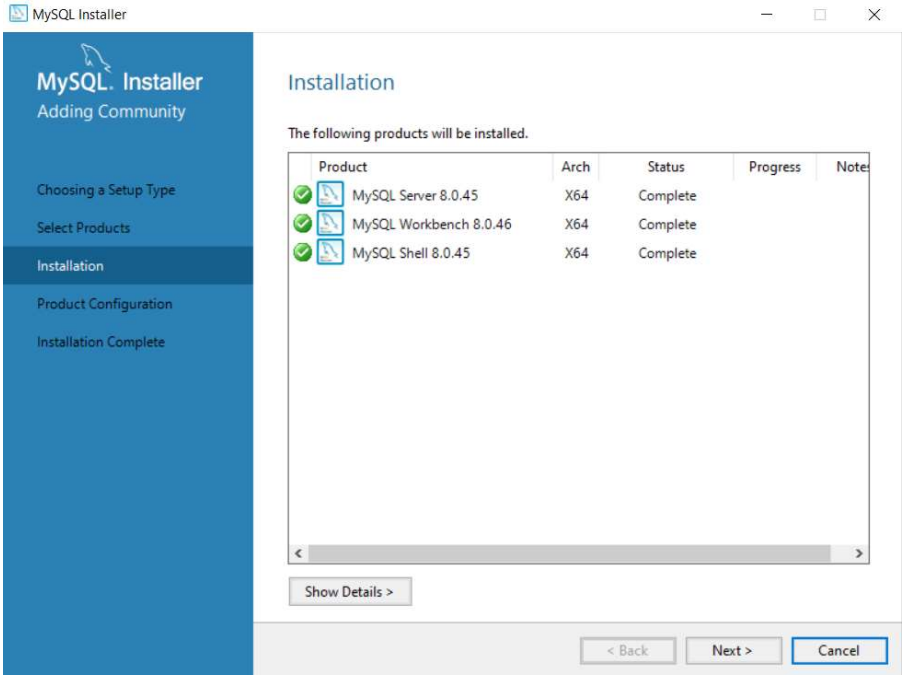
l. Click on next



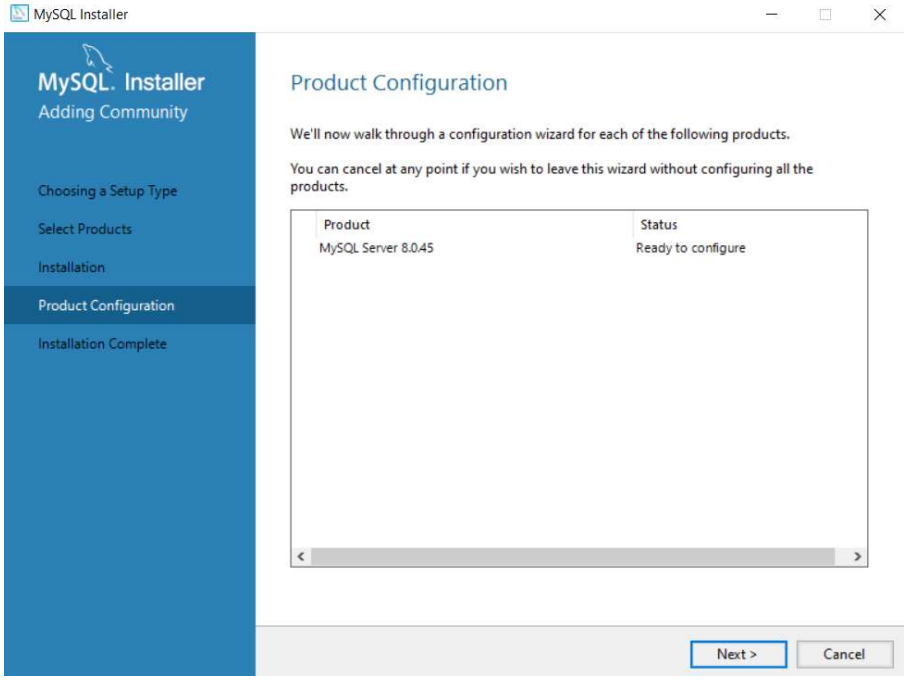
m. Click on execute



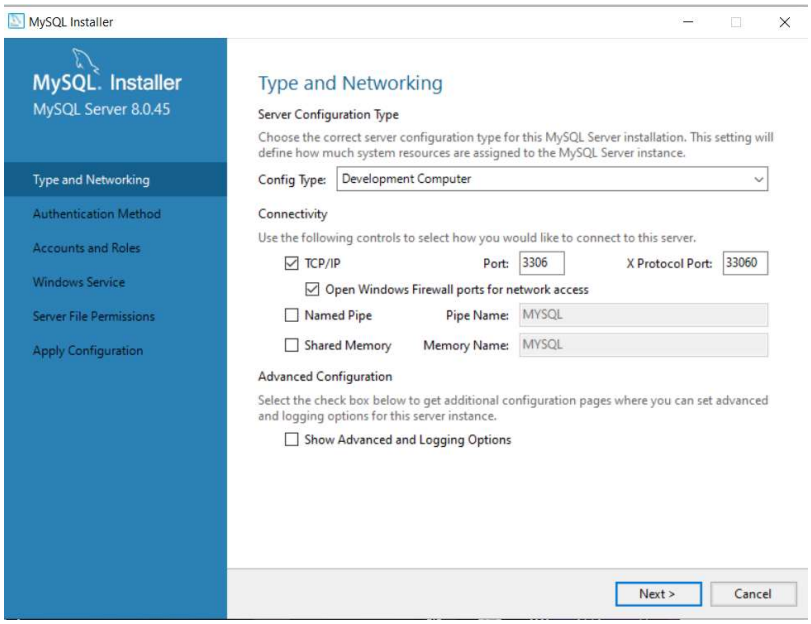
n. Click on next



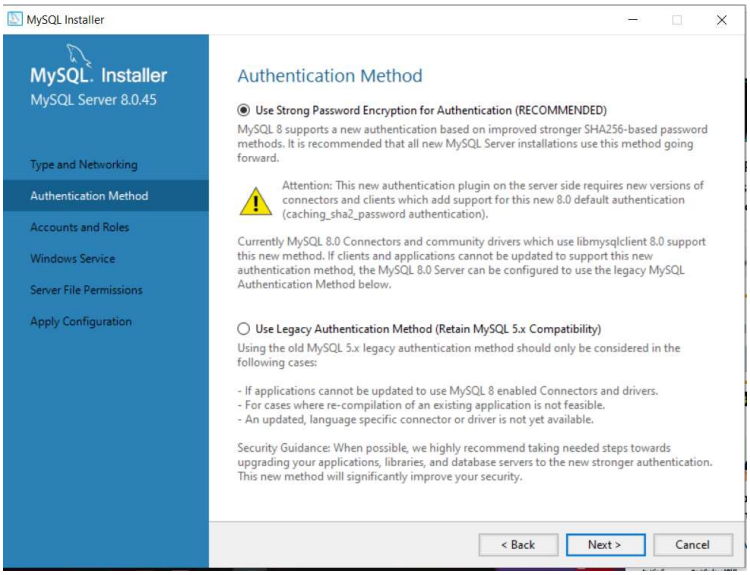
o. Click on next



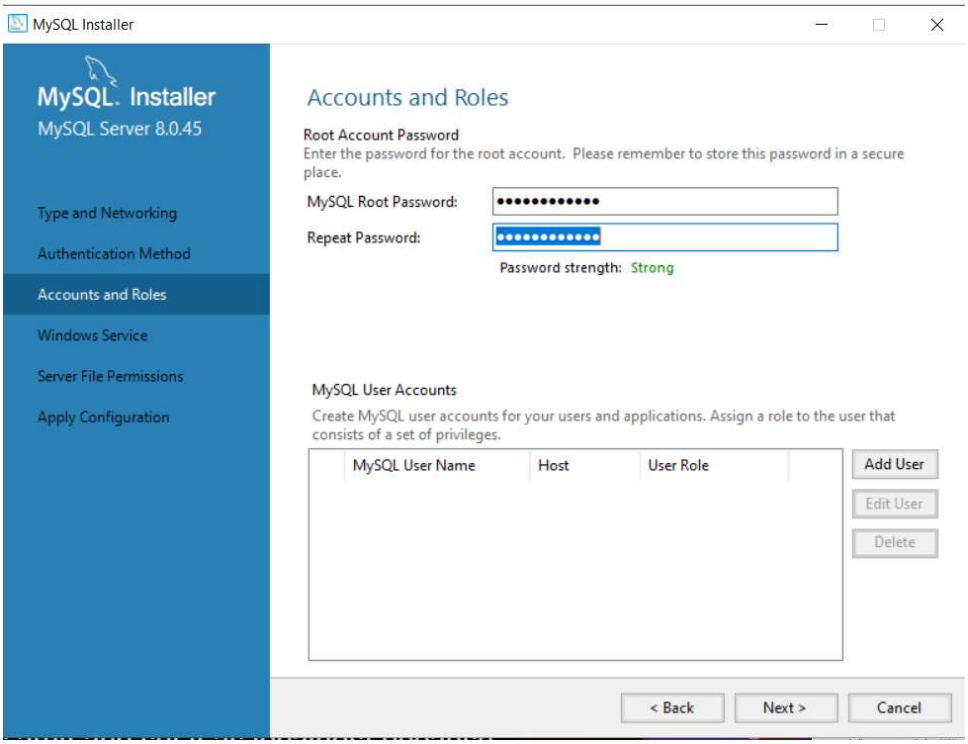
p. Click next



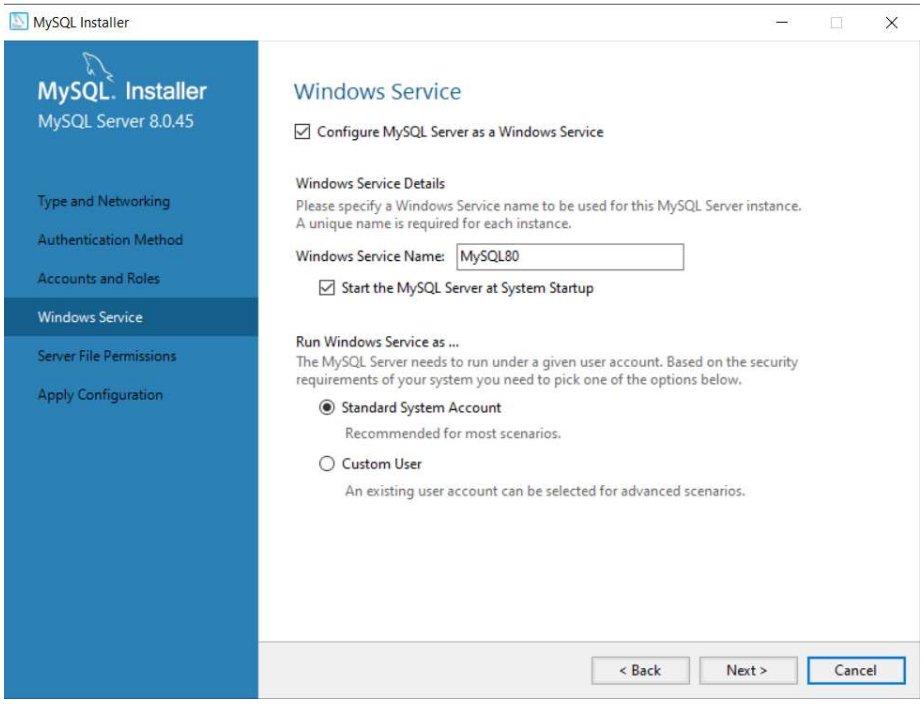
q. Click next



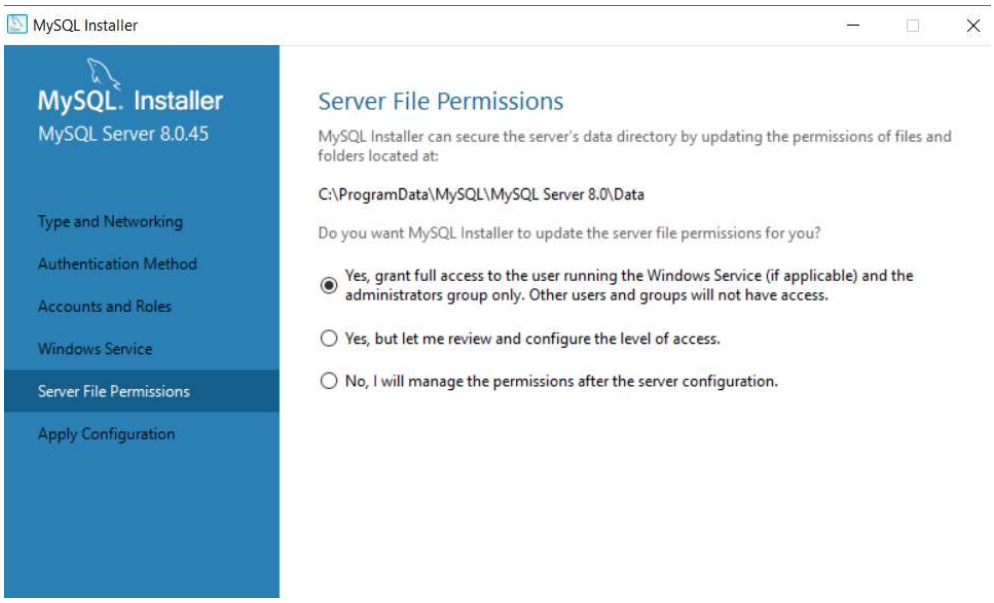
r. Set password and do next…..if you need to create user then
Click on new user {not mandatory} and add



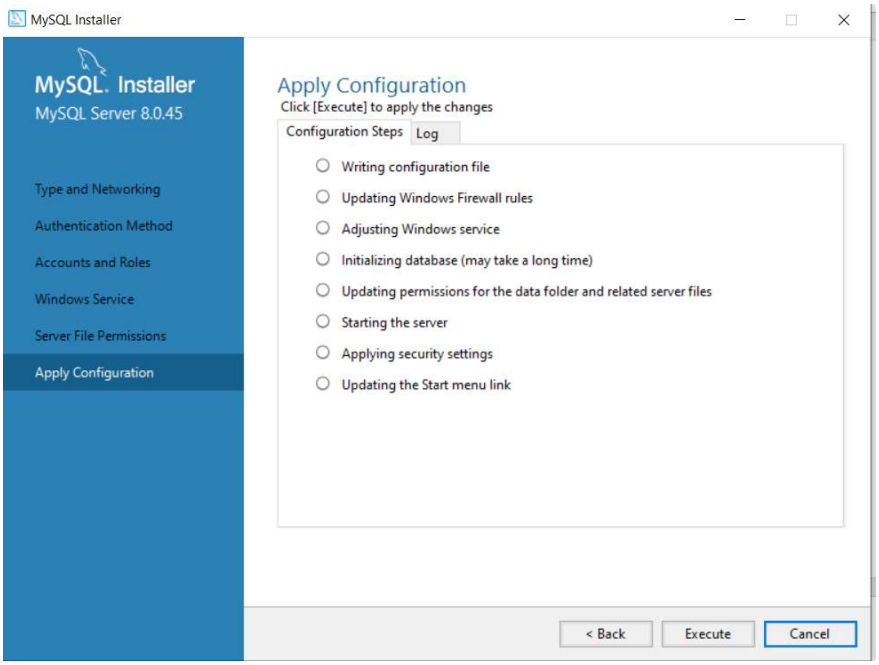
s. Click next



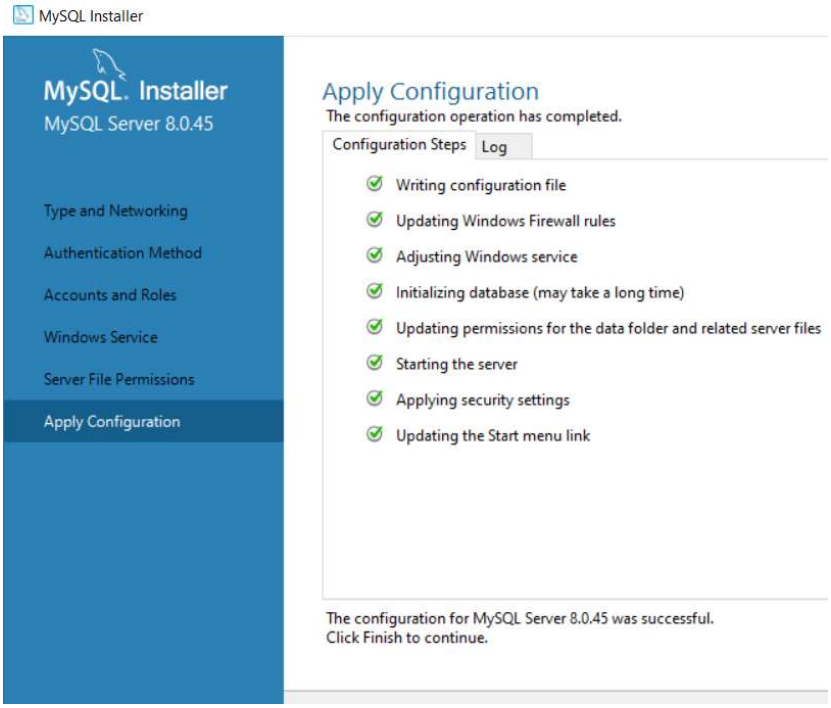
t. Click next



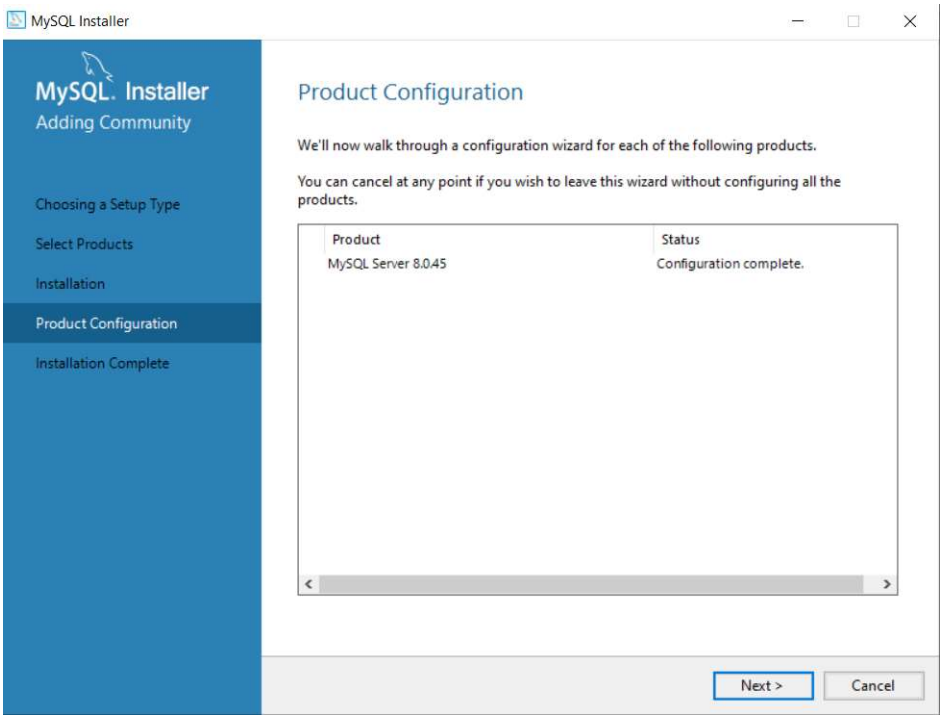
u. Click execute



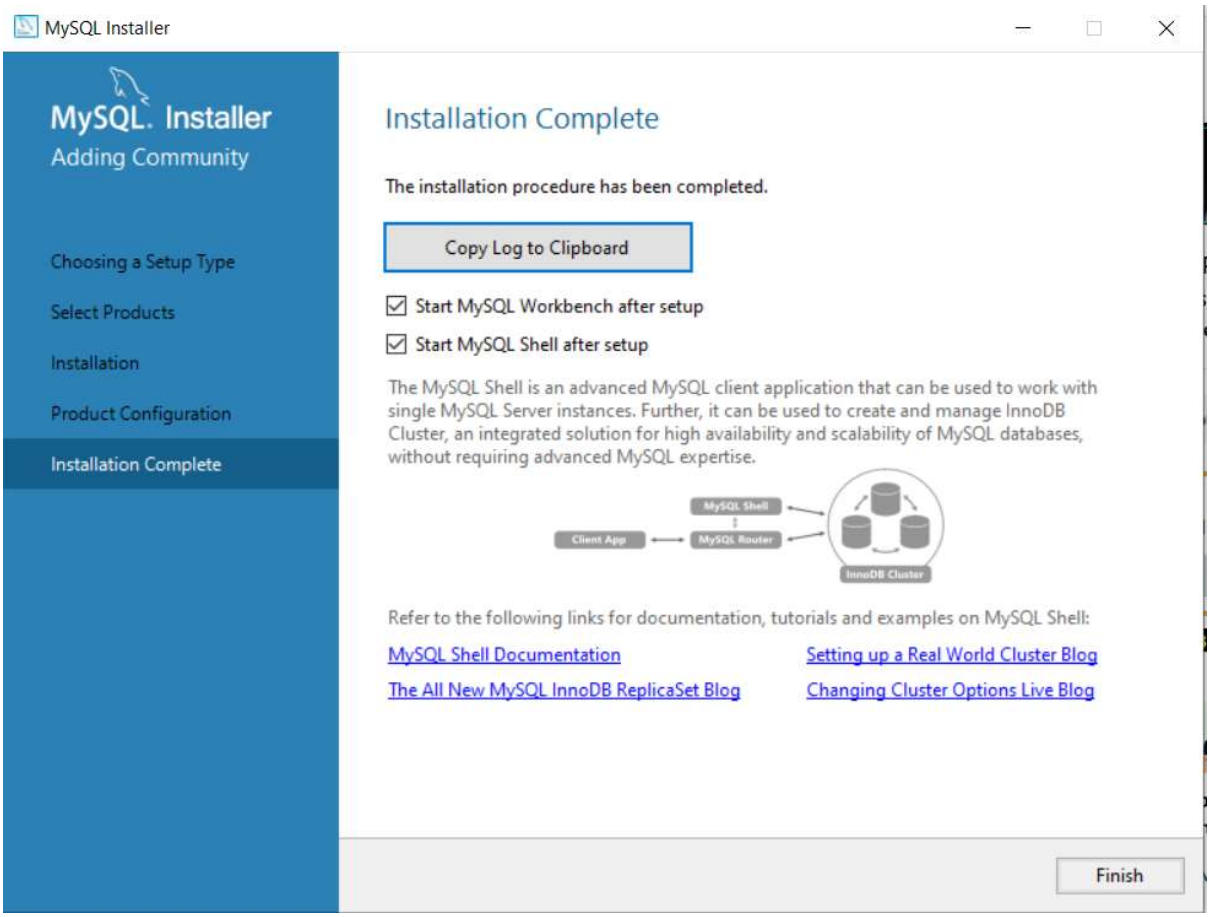
v. Click execute



w. Click finish



x. Click next

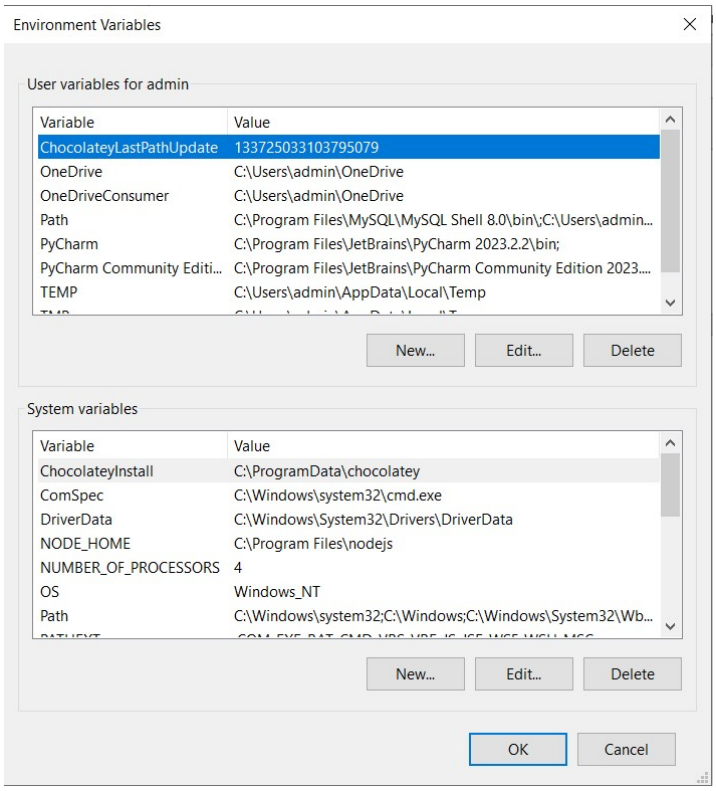


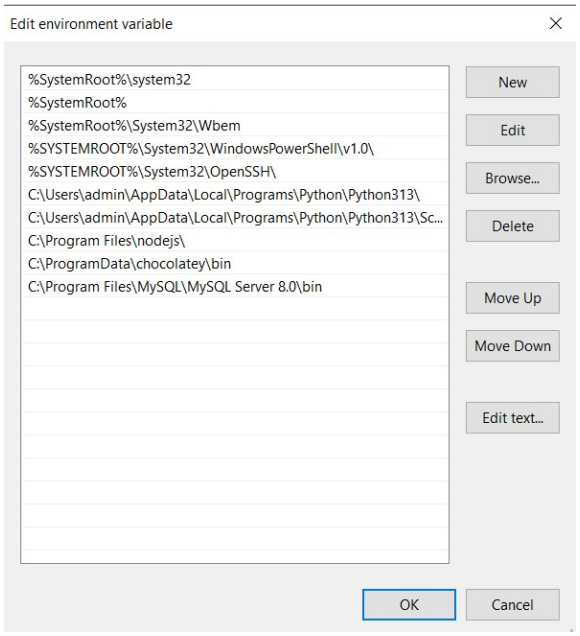
Set Environment Variables for MySql

Installed location →

C:\Program Files\MySQL\MySQL Server 8.0\bin

Go to edit system environment variable





Command Prompt

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>mysql --version
mysql  Ver 8.0.45 for Win64 on x86_64 (MySQL Community Server - GPL)

C:\Users\admin>
```

Mysql –u root –p

Command Prompt - mysql -u root -p

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin>mysql --version
mysql  Ver 8.0.45 for Win64 on x86_64 (MySQL Community Server - GPL)

C:\Users\admin>mysql -u root -p
Enter password: ************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

As per above screen, if it is visible then, mysql is successfully installed.

- **Show databases;**

It will show default database.

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.02 sec)

mysql>
```

Create a db → Create database pythondb;

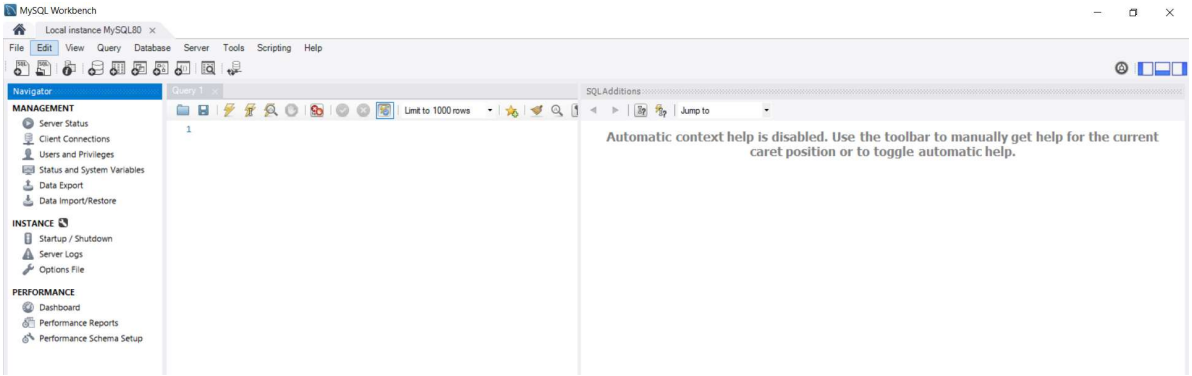
```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.02 sec)

mysql> create database pythondb;
Query OK, 1 row affected (0.02 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| pythondb           |
| sys                |
+--------------------+
5 rows in set (0.00 sec)

mysql>
```

Go to Mysql workbench

On bottom left side you will see

Click on root, it will ask for password , Write password and do next.

It will appear like




```
C:\Users\admin>pip --version
pip 25.2 from C:\Users\admin\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)

C:\Users\admin>
```

- Check pip version


```
C:\Users\admin>pip --version
pip 25.2 from C:\Users\admin\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)

C:\Users\admin>
```

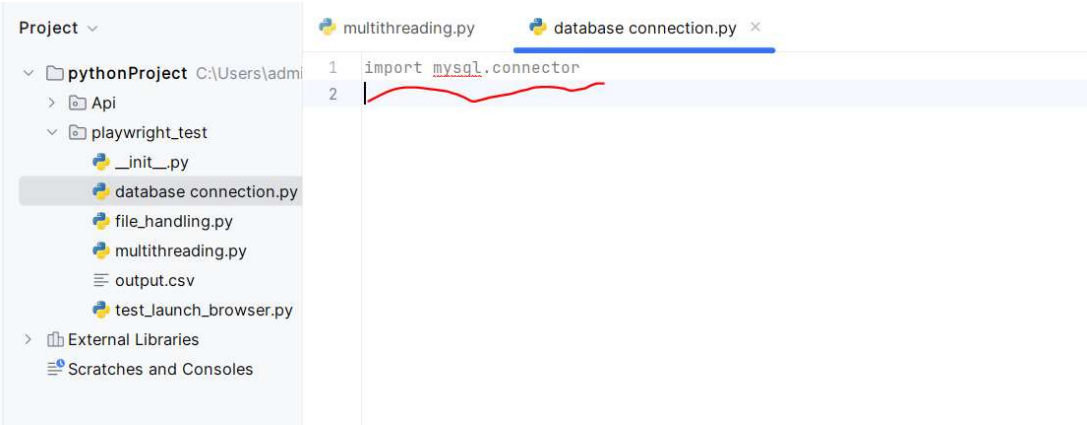

- Check pip list, It will show all installed packages.

Command Prompt

```
idna                3.10
iniconfig           2.0.0
numpy               2.1.2
outcome             1.3.0.post0
packaging           24.1
paramiko            3.5.0
pip                 25.2
playwright          1.53.0
pluggy              1.5.0
pycparser           2.22
pyee                13.0.0
PyNaCl              1.5.0
PySocks             1.7.1
pytest              8.3.3
pytest-base-url     2.1.0
pytest-playwright   0.7.0
python-slugify      8.0.4
requests            2.32.4
selenium            4.25.0
sniffio             1.3.1
sortedcontainers    2.4.0
text-unidecode      1.3
trio                0.27.0
trio-websocket      0.11.1
typing_extensions   4.12.2
urllib3             2.2.3
websocket-client    1.8.0
wsproto             1.2.0

C:\Users\admin>
```

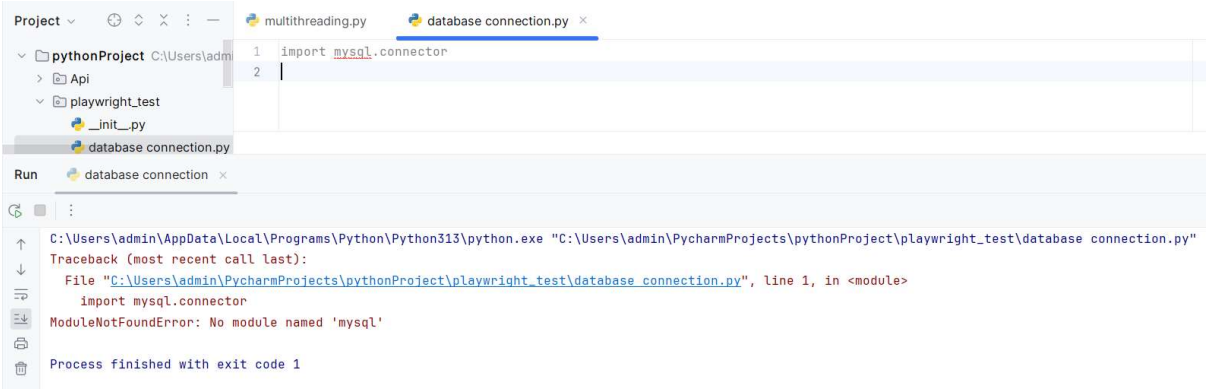
MySql package is not listed.

- Open any editor and create.py file and try to import mysql



Mqsql is having red tag, It means it is not available for use because it is not available.

Run the code as well, it will throw error.



We need to install mysql package for connection

- pip install mysql-connector-python

```
C:\Users\admin>pip install mysql-connector-python
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.6.0-cp313-cp313-win_amd64.whl.metadata (11 kB)
Downloading mysql_connector_python-9.6.0-cp313-cp313-win_amd64.whl (16.5 MB)
   ---------------------------------------- 16.5/16.5 MB 9.1 MB/s  0:00:01
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.6.0
```

Now Refresh the editor, error should not there.

Write a code to establish the connection?

```
import mysql.connector
conn = mysql.connector.connect(host='localhost', password = 'Prateek@1609',
user = 'root')

if conn.is_connected():
    print("Connection established")
```

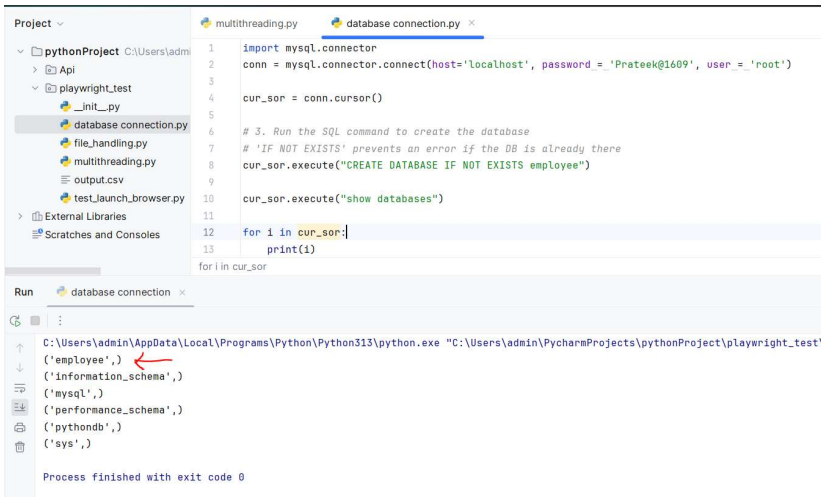
**

```python
import mysql.connector
conn = mysql.connector.connect(host='localhost', password = 'Prateek@1609',
user = 'root')

cur_sor = conn.cursor()
cur_sor.execute("show databases")

for i in cur_sor:
    print(i)
```

- **`conn` (Connection)**: The tunnel between your Python script and the MySQL server.
- **`cursor`**: A small robot you send through that tunnel to run a command and bring back the answer.

Create Database Employee

```python
import mysql.connector
conn = mysql.connector.connect(host='localhost', password = 'Prateek@1609',
user = 'root')

cur_sor = conn.cursor()

# 3. Run the SQL command to create the database
# 'IF NOT EXISTS' prevents an error if the DB is already there
cur_sor.execute("CREATE DATABASE IF NOT EXISTS employee")

cur_sor.execute("show databases")

for i in cur_sor:
    print(i)
```



Insert into database

```python
import mysql.connector

# 1. Connect specifically to the 'employee' database
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='Prateek@1609',
    database='employee'  # <--- Essential!
)

cursor = conn.cursor()

# 2. Create a 'staff' table if it doesn't exist
cursor.execute("""
    CREATE TABLE IF NOT EXISTS staff (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100),
        role VARCHAR(100),
        salary INT
    )
""")

# 3. Prepare the SQL and the data
sql = "INSERT INTO staff (name, role, salary) VALUES (%s, %s, %s)"
val = [
    ('Prateek', 'Manager', 85000),
    ('Aditi', 'Developer', 75000),
    ('Rahul', 'Designer', 60000)
]

# 4. Execute and COMMIT (This saves the data permanently)
cursor.executemany(sql, val)
conn.commit()

print(f"{cursor.rowcount} records inserted successfully.")

# 5. Clean up
cursor.close()
conn.close()
```

Ques:

Create a table student, Insert data and fetch some data and close the connection?

(This example, We are creating database first then creating table inside database)

```
import mysql.connector

# 1. Connect to the SERVER (no database specified yet)
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='Prateek@1609'
)
cursor = conn.cursor()

# 2. Create the DATABASE if it doesn't exist
cursor.execute("CREATE DATABASE IF NOT EXISTS school_db")
print("Database 'school_db' created or already exists.")

# 3. Switch to the database
cursor.execute("USE school_db")

# 4. Create the 'student' table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS student (
        roll_no INT PRIMARY KEY,
        first_name VARCHAR(50),
        last_name VARCHAR(50),
        phone_no VARCHAR(15),
        class VARCHAR(10)
    )
""")
print("Table 'student' is ready.")

# 5. Insert 5 students (using IGNORE to avoid errors if rerunning the
script)
sql = "INSERT IGNORE INTO student (roll_no, first_name, last_name,
phone_no, class) VALUES (%s, %s, %s, %s, %s)"
students = [
    (101, 'Amit', 'Sharma', '9876543210', '10th'),
    (102, 'Priya', 'Verma', '8765432109', '10th'),
    (103, 'Sohan', 'Khan', '7654321098', '11th'),
    (104, 'Riya', 'Singh', '6543210987', '12th'),
    (105, 'Vikram', 'Joshi', '5432109876', '11th')
]

cursor.executemany(sql, students)
conn.commit()
print(f"Check: {cursor.rowcount} new records added.")
```

```python
# 6. Fetch and display data
print("\n--- Student List ---")
cursor.execute("SELECT first_name, roll_no, class FROM student")

for (first_name, roll_no, student_class) in cursor:
    print(f"Name: {first_name} | Roll No: {roll_no} | Class:
{student_class}")

# 7. Close connections
cursor.close()
conn.close()
```

```
C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe C:\Users\admin\PycharmProjects\pythonProject\playwright_test\database.py
Database 'school_db' created or already exists.
Table 'student' is ready.
Check: 5 new records added.

--- Student List ---
Name: Amit | Roll No: 101 | Class: 10th
Name: Priya | Roll No: 102 | Class: 10th
Name: Sohan | Roll No: 103 | Class: 11th
Name: Riya | Roll No: 104 | Class: 12th
Name: Vikram | Roll No: 105 | Class: 11th

Process finished with exit code 0
```

1. `school_db` (The Database)

This is the **container** or "folder." It's the top-level storage area where all related information for your school project is kept. You can have many different tables (like students, teachers, exams) inside this one database.

2. `student` (The Table)

This is the **spreadsheet** or "file" inside that folder. It's a structured grid where:

- **Columns** are the categories (first name, roll no, etc.).
- **Rows** are the individual records (Amit, Priya, etc.).

The `cursor` object is your "worker" that carries messages between Python and MySQL. Here are the most common commands you will use:

1. The "Action" Commands

These send your SQL instructions to the database.

- `cursor.execute(query, params)` : Runs a **single** SQL statement. Use this for creating tables, deleting one row, or selecting data.
- `cursor.executemany(query, list_of_tuples)` : Runs the same command for **multiple** rows at once (like adding 5 students in one go). It is much faster than a loop.
- `cursor.callproc(proc_name, args)` : Used if you have "Stored Procedures" (pre-written scripts) saved inside your MySQL database.

2. The "Fetch" Commands

After you run a `SELECT` query, the cursor holds the results. You use these to pull that data into Python variables:

- `cursor.fetchone()` : Returns the **next single row** as a tuple. If no more rows exist, it returns `None`.
- `cursor.fetchall()` : Returns **all remaining rows** as a list of tuples. (Careful: if your table has 1 million rows, this might crash your RAM!)
- `cursor.fetchmany(size)` : Returns a specific number of rows (e.g., `cursor.fetchmany(10)` to get the next 10 students).

3. The "Information" Commands

These tell you what happened after an execution.

- `cursor.rowcount` : Returns the number of rows affected by the last command (e.g., "3 rows deleted" or "5 rows inserted").
- `cursor.lastrowid` : Returns the ID of the last row inserted (useful if you use `AUTO_INCREMENT`).

4. The "Cleanup" Command

- `cursor.close()` : Closes the cursor and frees up the database's memory. It’s good practice to do this once you're done with your queries.

Ques:

Create a table student, Insert data and fetch some data and close the connection?

(Same question as above, Here we are using database which is already given.)

```python
import mysql.connector

# 1. Connect to the database
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='Prateek@1609',
    database='school_db'  # Ensure this database exists first!
)

cursor = conn.cursor()

# 2. Create the 'student' table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS student (
        roll_no INT PRIMARY KEY,
        first_name VARCHAR(50),
        last_name VARCHAR(50),
        phone_no VARCHAR(15),
        class VARCHAR(10)
    )
""")
print("Table 'student' is ready.")

# 3. Insert data for 5 students
sql = "INSERT INTO student (roll_no, first_name, last_name, phone_no,
class) VALUES (%s, %s, %s, %s, %s)"
students = [
    (101, 'Amit', 'Sharma', '9876543210', '10th'),
    (102, 'Priya', 'Verma', '8765432109', '10th'),
    (103, 'Sohan', 'Khan', '7654321098', '11th'),
    (104, 'Riya', 'Singh', '6543210987', '12th'),
    (105, 'Vikram', 'Joshi', '5432109876', '11th')
]

# Use executemany for efficiency
cursor.executemany(sql, students)
conn.commit()  # Save changes!
print(f"{cursor.rowcount} students added.")

# 4. Fetch and display the data
print("\n--- Student List ---")
cursor.execute("SELECT first_name, roll_no, class FROM student")

for row in cursor.fetchall():
    print(f"Name: {row[0]} | Roll No: {row[1]} | Class: {row[2]}")

# 5. Close everything
cursor.close()
conn.close()
```

Ques

I want one more student entry

```
import mysql.connector

# 1. Connect to your database
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='Prateek@1609',
    database='school_db'
)
cursor = conn.cursor()

# 2. Add ONE new student
# Use %s placeholders for safety
new_student_sql = "INSERT IGNORE INTO student (roll_no, first_name,
last_name, phone_no, class) VALUES (%s, %s, %s, %s, %s)"
new_student_data = (106, 'Suresh', 'Raina', '4321098765', '12th')

cursor.execute(new_student_sql, new_student_data)

# 3. COMMIT the change to save it permanently
conn.commit()
print("New student added successfully!")

# 4. FETCH all students to see the result
print("\n--- Updated Student List ---")
cursor.execute("SELECT * FROM student")

# fetchall() gets all rows as a list
all_students = cursor.fetchall()

for row in all_students:
    print(row)

# 5. Close
cursor.close()
conn.close()
```

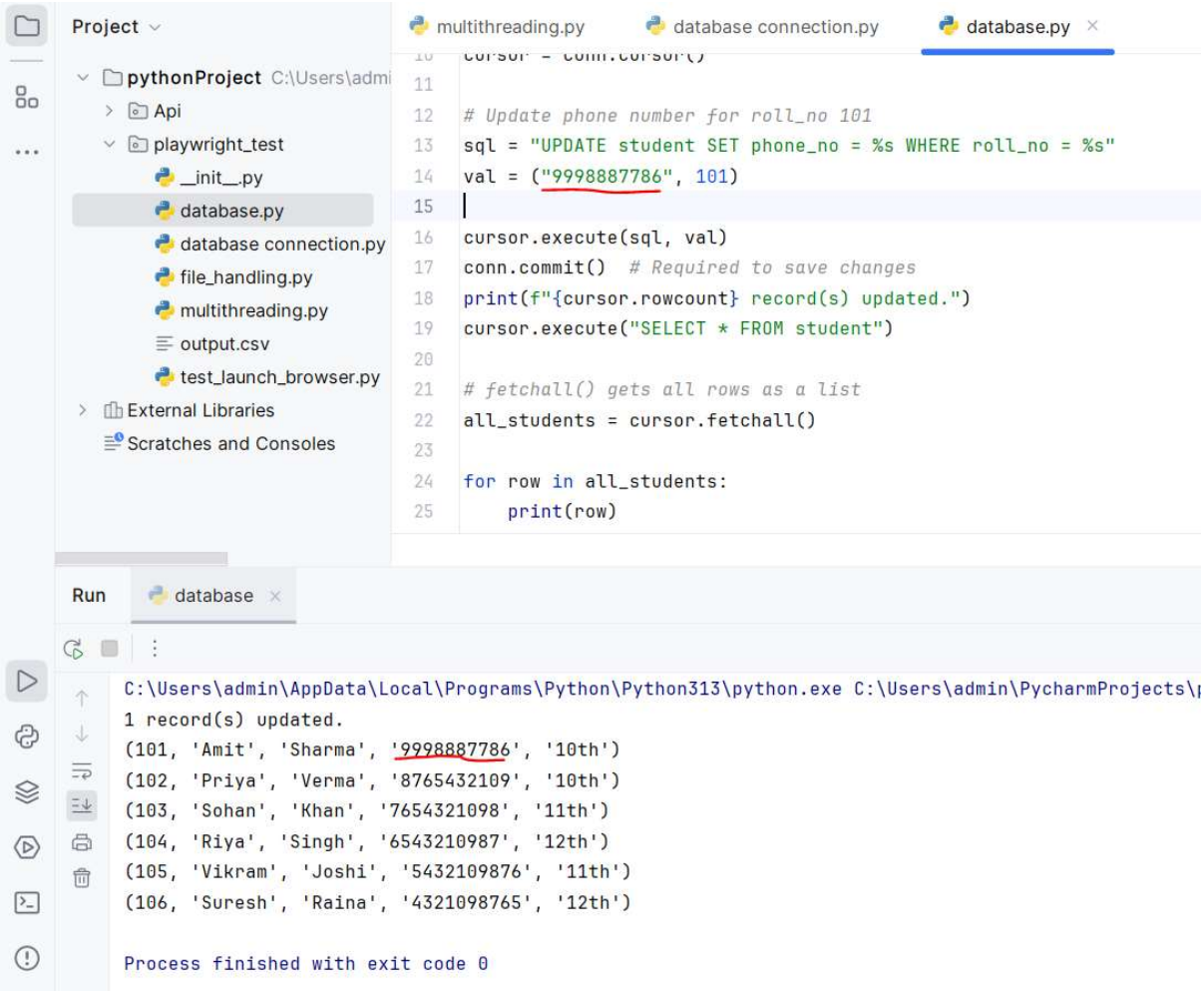
```
Run    database
New student added successfully!

--- Updated Student List ---
(101, 'Amit', 'Sharma', '9876543210', '10th')
(102, 'Priya', 'Verma', '8765432109', '10th')
(103, 'Sohan', 'Khan', '7654321098', '11th')
(104, 'Riya', 'Singh', '6543210987', '12th')
(105, 'Vikram', 'Joshi', '5432109876', '11th')
(106, 'Suresh', 'Raina', '4321098765', '12th')

Process finished with exit code 0
```

1. Update Data

Use the `UPDATE` command to modify existing records. **Always include a `WHERE` clause** to specify which record to change, or you will update every row in the table.

```python
import mysql.connector

# 1. Connect to your database
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='Prateek@1609',
    database='school_db'
)
cursor = conn.cursor()

# Update phone number for roll no 101
sql = "UPDATE student SET phone_no = %s WHERE roll_no = %s"
val = ("9998887776", 101)

cursor.execute(sql, val)
conn.commit()  # Required to save changes
print(f"{cursor.rowcount} record(s) updated.")
cursor.execute("SELECT * FROM student")

# fetchall() gets all rows as a list
all_students = cursor.fetchall()

for row in all_students:
    print(row)
```

```python
# 5. Close
cursor.close()
conn.close()
```



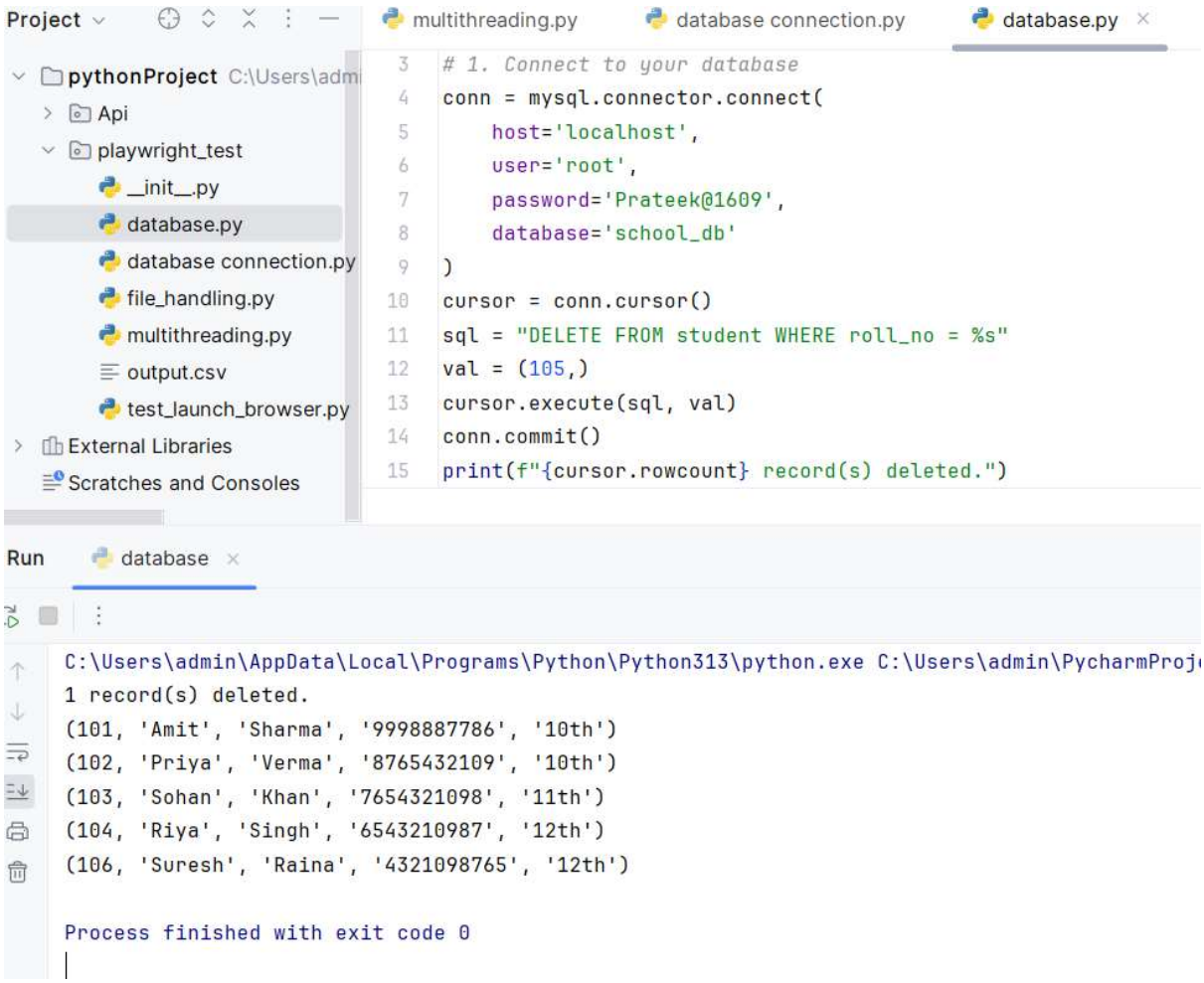
2. Delete Data

Use the `DELETE` command to remove records. Like updates, use a `WHERE` clause to avoid deleting all your data.

```python
import mysql.connector

# 1. Connect to your database
conn = mysql.connector.connect(
    host='localhost',
    user='root',
```

```python
    password='Prateek@1609',
    database='school_db'
)
cursor = conn.cursor()
sql = "DELETE FROM student WHERE roll_no = %s"
val = (105,)
cursor.execute(sql, val)
conn.commit()
print(f"{cursor.rowcount} record(s) deleted.")
cursor.execute("SELECT * FROM student")
# fetchall() gets all rows as a list
all_students = cursor.fetchall()

for row in all_students:
    print(row)
# 5. Close
cursor.close()
conn.close()
```



3. Filter Data (WHERE Clause)

Instead of fetching everything, you can search for specific groups, such as all students in the '10th' class.

```
cursor.execute("SELECT * FROM student WHERE class = '10th'")
for row in cursor.fetchall():
    print(row)
```

4. Sort and Limit Results

You can organize your data or only show the first few results.

- **Order By**: `SELECT * FROM student ORDER BY first_name ASC` (Sorts alphabetically).
- **Limit**: `SELECT * FROM student LIMIT 3` (Shows only the first 3 students).

Ques

Python & MySQL Assignment: Employee Management

Objective: Write a Python script to manage employee records in a MySQL database.

Task Requirements:

1. **Database Creation**: Create a database named `company_db`.
2. **Table Creation**: Inside `company_db`, create a table named `employees` with the following columns:

1. `emp_id` (Integer, Primary Key)
2. `name` (Varchar 50)
3. `department` (Varchar 30)
4. `salary` (Float)

3. **Data Entry**: Insert records for **3 employees** using a single `executemany()` command.

0. *Example data: (1, 'Arjun', 'IT', 75000.0), (2, 'Sara', 'HR', 60000.0), (3, 'Vikram', 'Sales', 55000.0)*

4. **Data Retrieval**: Fetch and print only the **Name** and **Salary** of all employees whose salary is **greater than 58,000**.

5. **Cleanup**: Ensure you **commit** your changes and **close** both the cursor and the connection

Solution

<----------->

```
# Create Table
cursor.execute("CREATE TABLE IF NOT EXISTS employees (emp id INT PRIMARY
KEY, name VARCHAR(50), department VARCHAR(30), salary FLOAT)")

# Insert Multiple
sql = "INSERT IGNORE INTO employees (emp_id, name, department, salary)
VALUES (%s, %s, %s, %s)"
data = [(1, 'Arjun', 'IT', 75000.0), (2, 'Sara', 'HR', 60000.0), (3,
'Vikram', 'Sales', 55000.0)]
cursor.executemany(sql, data)
conn.commit()

# Filtered Fetch
cursor.execute("SELECT name, salary FROM employees WHERE salary > 58000")
for (name, salary) in cursor.fetchall():
    print(f"Employee: {name}, Salary: {salary}")
```

As per above question, implement below as

update 1: The Promotion (Update)

- **Task:** The employee with `emp_id = 2` has been promoted. Change their **Salary** to **65,000.0** and their **Department** to **'Senior HR'**.
- **Check:** Ensure they use the `UPDATE` command and don't forget to `conn.commit()`.

update 2: The Departure (Delete)

- **Task:** Employee **'Vikram'** has left the company. Remove his record from the `employees` table using his name.
- **Check:** Remind them to use `WHERE name = 'Vikram'` to avoid deleting everyone!

update 3: Total Count (Information)

- **Task:** Use `cursor.rowcount` or `COUNT(*)` to print the total number of employees remaining in the table.

Solution

- ```
  # Bonus 1: Update Sara's details
  sql_update = "UPDATE employees SET salary = %s, department = %s WHERE emp_id = %s"
  val_update = (65000.0, 'Senior HR', 2)
  cursor.execute(sql_update, val_update)
  conn.commit()

  # Bonus 2: Delete Vikram
  sql_delete = "DELETE FROM employees WHERE name = %s"
  val_delete = ("Vikram",)
  cursor.execute(sql_delete, val_delete)
  conn.commit()

  # Bonus 3: Count remaining
  cursor.execute("SELECT COUNT(*) FROM employees")
  total = cursor.fetchone()[0]
  print(f"Total employees remaining: {total}")
  ```
  ```